

Desarrollo de Sistemas Informáticos - Grado en Ingeniería Informática - ULL

[View project on GitHub](#)

Segundo trabajo en grupo - MedCore: API REST con Node/Express

En este segundo trabajo en grupo tendrá que implementar un API REST, haciendo uso de Node/Express, que permita llevar a cabo operaciones de creación, lectura, modificación y borrado (Create, Read, Update, Delete - CRUD) para gestionar el sistema de información de un hospital de tamaño mediano que atiende a pacientes de toda la región.

Todo el código desarrollado deberá estar alojado en el repositorio generado tras la aceptación de la asignación grupal de GitHub Classroom. En ese sentido, utilice en dicho repositorio una estructura de proyecto similar a la que hemos visto en clase.

Algunas tareas previas

Acepte la **asignación de GitHub Classroom** asociada a esta práctica. Tenga en cuenta que es una asignación grupal. Deben respetarse los equipos seleccionados en el aula virtual y, por lo tanto, que desarrollaron el primer trabajo en grupo (en la medida de lo posible).

Deberá utilizar **MongoDB/MongoDB Atlas** como sistema de base de datos no relacional, además de **Mongoose**, para gestionar la base de datos desde Node.js. Además, utilice las opciones y validadores que estime oportunos en sus esquemas.

Deberá desplegar el API en **Render**. En las últimas sesiones de teoría, veremos un ejemplo sencillo que ilustrará lo anterior.

Contexto del sistema

El Hospital Universitario de la Costa ha decidido modernizar su infraestructura tecnológica y necesita un API REST que sirva como núcleo de su nuevo sistema de información. Actualmente, el hospital gestiona su actividad mediante hojas de cálculo dispersas y registros en papel, lo que genera errores frecuentes: pacientes que reciben medicación incorrecta por falta de información sobre alergias, citas duplicadas, o pérdida de historial clínico al dar de alta a un médico.

El API que desarrollará centralizará cuatro grandes áreas: la gestión de pacientes, la gestión del personal médico, el catálogo de medicamentos y los registros de consultas o ingresos. Todas ellas están interrelacionadas y requieren una lógica de negocio cuidadosa para garantizar la integridad de los datos.

Descripción de los requisitos del API

Pacientes

Cualquier persona que acude al hospital debe estar registrada en el sistema antes de que se le pueda asignar una consulta o un ingreso. Un paciente puede ser un adulto autónomo, un menor con tutor legal, o un paciente crónico con múltiples visitas previas.

En la ruta `/patients` del API se deberá poder crear, leer, actualizar o borrar un paciente a través de los métodos HTTP necesarios.

Piense en los campos de información que permiten modelar un paciente. Como mínimo, debería considerar:

- Nombre completo y fecha de nacimiento (a partir de la cual podría calcularse la edad).
- Número de identificación (DNI, pasaporte u otro documento), que deberá ser único en el sistema.
- Número de la seguridad social o número de historial clínico, también único.
- Género.
- Datos de contacto: dirección, teléfono y correo electrónico.
- Alergias conocidas (puede ser una lista vacía, pero el campo debe existir).
- Grupo sanguíneo, con validación sobre los valores permitidos (A+, A-, B+, B-, AB+, AB-, O+, O-).
- Estado actual del paciente: activo, de baja temporal, o fallecido.

Recuerde utilizar operadores y validadores en el esquema correspondiente, como restricciones de longitud mínima, formato de correo electrónico o enumeraciones para los campos con valores acotados.

La operación de lectura o consulta podrá llevarse a cabo de dos maneras: o bien mediante una query string en la que se consulte por el nombre completo o por el número de identificación del paciente, o bien mediante el identificador único asignado por la base de datos como parámetro dinámico en la URL. Las operaciones de modificación y borrado también podrán realizarse de ambos modos.

Teniendo en cuenta lo anterior, como mínimo deberá escribir un total de siete manejadores para esta ruta.

Consideración especial: al borrar un paciente del sistema, deberá decidir qué ocurre con sus registros de consultas e ingresos asociados. Justifique su decisión en la documentación.

Personal médico

El hospital cuenta con médicos de distintas especialidades, así como con personal de enfermería y otros especialistas sanitarios. Todos ellos pueden estar asignados como responsables de una consulta o ingreso.

En la ruta `/staff` del API se deberá poder crear, leer, actualizar o borrar un miembro del personal médico a través de los métodos HTTP necesarios.

Como mínimo, considere los siguientes campos:

- Nombre completo.
- Número de colegiado, que deberá ser único en el sistema.
- Especialidad médica (por ejemplo: medicina general, cardiología, traumatología, pediatría, oncología, urgencias). Utilice una enumeración para validar los valores permitidos, pero permita que el equipo la amplíe de manera razonada.
- Categoría profesional: médico/a adjunto/a, médico/a residente, enfermero/a, auxiliar de enfermería, jefe/a de servicio.
- Turno de trabajo habitual: mañana, tarde, noche o rotatorio.
- Número de consulta o planta asignada habitualmente.
- Años de experiencia.
- Datos de contacto del departamento.

Estado: activo o inactivo (por ejemplo, en excedencia o baja laboral).

La operación de lectura podrá realizarse mediante una query string por nombre o especialidad, o mediante el identificador único como parámetro dinámico. Las operaciones de modificación y borrado también podrán realizarse de ambos modos.

Como mínimo deberá escribir un total de siete manejadores para esta ruta.

Consideración especial: si un miembro del personal médico se elimina del sistema, ¿qué ocurre con las consultas e ingresos en los que figuraba como responsable? Justifique su decisión.

Medicamentos

El hospital dispone de una farmacia interna que gestiona el stock de medicamentos disponibles para ser prescritos durante las consultas o ingresos. Es imprescindible mantener actualizado el inventario para evitar prescripciones imposibles de dispensar.

En la ruta `/medications` del API se deberá poder crear, leer, actualizar o borrar un medicamento a través de los métodos HTTP necesarios.

Como mínimo, considere los siguientes campos:

Nombre comercial y nombre del principio activo, ambos con validación de longitud mínima.

Código nacional del medicamento, único en el sistema.

Forma farmacéutica: comprimido, cápsula, solución oral, solución inyectable, pomada, parche transdérmico, inhalador u otras.

Dosis estándar por toma (por ejemplo, 500 mg) y unidad de medida.

Vía de administración: oral, intravenosa, intramuscular, subcutánea, tópica, inhalatoria.

Stock disponible en unidades, con validación de valor mínimo igual a cero.

Precio por unidad (en euros), con validación de valor mínimo.

Requiere prescripción médica: booleano.

Fecha de caducidad del lote en stock.

Contraindicaciones conocidas (lista de cadenas de texto).

La operación de lectura podrá realizarse mediante una query string que permita buscar por nombre comercial, por principio activo o por código nacional (de forma individual o combinada), o bien mediante el identificador único como parámetro dinámico. Las operaciones de modificación y borrado también podrán realizarse de ambos modos.

Como mínimo deberá escribir un total de siete manejadores para esta ruta.

Consideración especial: antes de borrar un medicamento del catálogo, ¿qué debería comprobarse en relación a los registros de consultas que lo referencian?

Consultas e ingresos

Esta es la entidad central del sistema, ya que representa el acto médico en sí. Una consulta puede ser una visita ambulatoria de pocos minutos o un ingreso hospitalario de varios días. En ambos casos, existe un paciente atendido, un médico responsable, y una serie de medicamentos prescritos.

En la ruta `/records` del API se deberá poder crear, leer, actualizar o borrar registros médicos a través de los métodos HTTP necesarios.

Modelo de datos

Un registro médico debe contener, como mínimo:

Referencia al paciente (mediante su `_id` en la base de datos).

Referencia al médico responsable (mediante su `_id`).

Tipo de registro: consulta ambulatoria o ingreso hospitalario.

Fecha y hora de inicio, que se establecerá automáticamente en el momento de la creación si no se especifica.

Fecha y hora de alta o finalización (puede estar vacía si el ingreso sigue activo).

Motivo de consulta o ingreso.

Diagnóstico emitido.

Lista de medicamentos prescritos, donde cada elemento incluirá:

- Referencia al medicamento (mediante su `_id`).
- Cantidad prescrita en unidades.
- Posología (instrucciones de administración, como texto libre).

Importe total de la medicación prescrita, que deberá calcularse automáticamente a partir del precio unitario de cada medicamento y las cantidades prescritas.

Estado del registro: abierto (el paciente sigue ingresado o la consulta no ha finalizado) o cerrado.

Lógica de creación

La operación de creación de un registro médico recibirá en el cuerpo de la petición toda la información necesaria, entre la que deberán encontrarse:

El número de identificación del paciente (DNI u otro documento), no su `_id`.

El número de colegiado del médico responsable, no su `_id`.

Una lista de objetos con el código nacional de cada medicamento prescrito y la cantidad correspondiente.

La lógica del manejador deberá:

Verificar que el paciente existe en la base de datos buscando por su número de identificación.

Verificar que el médico existe y está activo buscando por su número de colegiado.

Para cada medicamento de la lista, verificar que existe en la base de datos usando su código nacional y que el stock disponible es suficiente para la cantidad prescrita.

Descontar del stock de cada medicamento la cantidad prescrita.

Calcular el importe total sumando el producto del precio unitario por la cantidad de cada medicamento.

Crear el registro con referencias a los `_id` de paciente, médico y medicamentos, junto al importe calculado y la fecha y hora actuales.

Dado que este manejador incorpora una lógica compleja, trate de organizar el código en funciones auxiliares bien nombradas para mejorar su legibilidad y mantenibilidad.

Lógica de lectura

La operación de consulta de registros médicos podrá realizarse de tres maneras distintas:

Mediante una query string que incluya el número de identificación del paciente, devolviendo todos sus registros ordenados cronológicamente.

Mediante una query string que permita filtrar por un rango de fechas (parámetros `startDate` y `endDate`) y, opcionalmente, por tipo de registro (consulta ambulatoria, ingreso o ambos).

Mediante el identificador único del registro como parámetro dinámico en la URL.

Lógica de modificación

La modificación de un registro solo podrá realizarse a través de su identificador único. Si se modifica la lista de medicamentos prescritos, la lógica deberá:

- Restaurar el stock de los medicamentos que figuraban en la versión anterior del registro.
- Verificar la disponibilidad de los nuevos medicamentos.
- Descontar el nuevo stock y recalcular el importe total.

Lógica de borrado

El borrado de un registro solo podrá realizarse a través de su identificador único. Al borrar un registro (interpretado como una cancelación o corrección), la lógica deberá restaurar el stock de todos los medicamentos que habían sido prescritos en él.

Teniendo en cuenta todo lo anterior, como mínimo deberá escribir un total de seis manejadores para esta ruta.

Requisitos transversales

Gestión de errores

Programe defensivamente y contemple al menos los siguientes escenarios de error:

- El paciente, médico o medicamento referenciado no existe en la base de datos.
- El stock de un medicamento es insuficiente para la cantidad prescrita.
- Se intenta prescribir un medicamento cuya fecha de caducidad ya ha pasado.
- Se intenta asignar como responsable a un médico con estado inactivo.
- Se accede a una ruta inexistente en el API.
- Se envía un cuerpo de petición con campos inválidos o ausentes.
- Se produce un error interno del servidor.

En todos los casos, devuelva códigos de estado HTTP apropiados (400 , 404 , 409 , 500 , etc.) junto a mensajes de error descriptivos en formato JSON.

Documentación

Documente el API haciendo uso de TSDoc para el código fuente y de una herramienta como Swagger/OpenAPI para describir los endpoints. La documentación deberá incluir, para cada ruta, los parámetros admitidos, el formato del cuerpo de la petición, los posibles códigos de respuesta y un ejemplo de respuesta exitosa.

Pruebas

Escriba pruebas automatizadas haciendo uso de Vitest. Como mínimo, deberá cubrir:

- La creación correcta de cada entidad.
- La creación con datos inválidos o incompletos.
- La lectura por query string y por parámetro dinámico.
- La modificación correcta e incorrecta.
- El borrado y sus efectos sobre entidades relacionadas.
- La creación de un registro médico con stock suficiente e insuficiente.farmacéutica

Integración continua

Configure un flujo de integración continua mediante GitHub Actions que ejecute automáticamente las pruebas, GitHub Code Quality y Coveralls en cada push o pull request a la rama principal.

Despliegue

Despliegue el API en Render conectado a una base de datos en MongoDB Atlas. El enlace al API desplegado deberá figurar en el `README.md` del repositorio, junto a instrucciones claras para ejecutar el proyecto en local.

Resumen de manejadores mínimos

Ruta	Manejadores mínimos
/patients	7
/staff	7
/medications	7
/records	6
Total	27

Entrega

Enlace al repositorio de GitHub con todo el código, la documentación y los resultados de las pruebas.
Enlace al API desplegado en Render.

grp02-medcore-api is maintained by ULL-ESIT-INF-DSI-2526.

This page was generated by [GitHub Pages](#).